
從第一原理理解 Kafka

從「為什麼需要它」到第一次讀懂 Kafka Issue

Version B · Short Main Path v1.1

Prototype for Review

Author	Yen-Hua Chen
Technical baseline	Apache Kafka 4.3.1
Language	zh-TW
License	CC BY 4.0

Original source:

<https://github.com/hikaru-212/open-source-from-first-principles>

版本範圍：本文以 Apache Kafka 4.3.1 為技術基準。現在的 Kafka server 使用 KRaft；ZooKeeper 僅屬於歷史與遷移議題，不放入初學主線。

文件定位與版本範圍

本文件為教學 prototype，以 Apache Kafka 4.3.1 為技術基準，目標是建立 Kafka 的核心系統心智模型，並提供從理解機制到閱讀 issue、test 與 source code 的最短入門路徑。

為控制初學者的認知負擔，本文刻意省略部分協定細節、歷史架構與進階機制；文中的簡化不應被視為 Kafka 完整規格。若本文描述與 Apache Kafka 對應版本的官方文件、Javadoc 或 source code 存在差異，應以官方資料為準。

Deep Dive 與進階實作細節將留待後續擴充版本補充。

Kafka 到底是什麼？

Kafka 是以可持久保存、分區的 log（durable partitioned log）為核心的分散式事件串流系統。

它讓持續產生的事件不必依賴收件端即時在線：producer 寫入 records 後，consumer 可在保留期間內，依各 partition 的順序與自己的進度持續讀取，不必與 producer 同時活著。

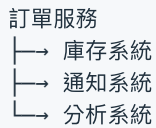
Kafka 負責事件流的持久保存、分區、partition 內排序與交付；它不負責證明 consumer 的外部 database write、HTTP effect 或業務流程已 exactly-once 完成。

有了這個定位，第一章再追問：為什麼 producer 與 consumer 不該必須同時活著？

Chapter 1 — Why Kafka?

如果 Producer 和 Consumer 必須同時活著，會怎樣？

假設訂單服務完成付款後，必須通知三個系統：



最直覺的做法，是讓訂單服務同步呼叫三個 API。

流量不大、所有服務都正常時，這個設計很簡單。但接著會出現幾個物理問題：

- 通知系統停機，訂單是否也要失敗？
- 分析系統慢十倍，訂單請求是否得陪它等待？
- 半年後加入風險偵測服務，它如何取得過去的訂單？
- 分析邏輯寫錯，需要重算昨天的資料時，原始事件還在嗎？

增加 timeout 和 retry 可以緩解部分症狀，卻沒有改變根本耦合：在這種同步直連設計裡，Producer 的成功路徑仍被 Consumer 當下的存活與速度綁住。

我們真正需要的是一個可以保留事件的中介：

訂單服務



可保留事件的中介



它至少要維持三項條件：

1. 事件寫入後，不因某個 Consumer 暫時離線而消失。
2. 每個 Consumer 可以依自己的速度前進。
3. 舊資料在保留期間內仍能重新讀取。

到這裡，我們才需要名稱：寫入資料的程式叫 **producer**，讀取資料的程式叫 **consumer**，Kafka 是中間負責保存與提供資料的系統。

初學時，可以把 Kafka 想成：

一組分散式、可持久保存的 log。

但這只是起點。Kafka 不是單一無限大的 log，也不會永遠保留所有資料。它保存的是多條可分割、可複製、受 retention 規則限制的 log。

Kafka 能證明某筆 record 已跨過特定的寫入與複製邊界，也能讓 Consumer 在保留期間內重新取得它；Kafka 不能因此證明庫存已扣除、通知已送達，或事件內容在商業上一定正確。

Checkpoint

如果分析系統離線兩小時，但訂單服務仍要繼續工作，系統必須保留哪些狀態，才能讓分析系統恢復後從自己的進度繼續，而不要求訂單服務重送所有資料？

Deep Dive（選讀）

- Retention policy 與儲存容量
- Kafka Connect、Kafka Streams 各自解決的問題

Chapter 2 — What Must Be Ordered Together?

到底哪些事件真的需要同一個順序？

假設我們希望所有事件都有清楚順序。最直覺的設計是把它們送進同一條 lane：



這確實得到一個總順序，但代價也很直接：每個 Producer 都必須經過同一個排序點。原本互不相關的工作，也被迫排隊等待。

台北訂單與高雄訂單可能完全獨立，卻仍要競爭同一條 lane。當流量增加，這個共享排序點會同時成為協調成本、序列化瓶頸與故障集中點。

因此，真正該問的不是：

如何讓所有事件都有同一個順序？

而是：

哪些事件真的必須一起排序？

例如，同一張訂單的狀態可能必須保持：

`created → paid → shipped`

但訂單 A 與訂單 B 未必需要互相比較先後。

Kafka 的設計選擇，是把一個 topic 分成多個 **partition**。每個 partition 是一個獨立的 ordering domain：

partition 0: A-created → A-paid → A-shipped
partition 1: B-created → B-paid → B-cancelled

Kafka 保證的是 partition 內的 log order；不同 partition 之間沒有一個 Kafka 提供的 total order。比較 `partition 0` 的 offset 100 與 `partition 1` 的 offset 80，不能推出誰在全域上先發生。

接著才需要 **key**。如果同一張訂單的事件需要留在同一個 ordering domain，`order_id` 就可能是合理的 key。Producer 的 partition-selection 規則會利用 key 決定資料送往哪個 partition。

但「相同 key 永遠在相同 partition」需要前提：

- 沒有明確指定另一個 partition；
- 沒有改用不同的 custom partitioner；
- key 沒有被設定成忽略；
- serializer 與 partition 數量維持相容。

尤其是增加 partition 數量後，未來相同 key 的映射可能改變。擴充分區不只是效能操作，也可能影響原本依賴的 ordering domain。

從 ordering 的角度看，Partition 最值得先理解的價值，不只是把資料切開，而是讓不同 ordering domains 可以獨立前進；真正需要共享順序的資料，才共同承擔排序成本。

Checkpoint

包裹追蹤系統中，每個包裹都有「收件、轉運、派送、簽收」事件。你會選擇什麼作為 key？這個選擇保證了哪一種順序，又刻意放棄了哪一種全域順序？

Deep Dive（選讀）

- Hot partition 與資料偏斜
- Custom partitioner
- 擴充分區時的 key migration

Chapter 3 — Key, Partition, Offset, Consumer Group

資料在哪條 lane？讀到哪裡？現在誰負責？

不要先背四個名詞。先回答三個物理問題。

問題一：這筆 record 屬於哪條 lane？

Producer 送出 record 時，partition-selection 規則會根據 key、明確指定的 partition 或其他設定，選出一個 partition。

Key 本身不是順序。它只是選擇 ordering domain 的輸入；真正提供順序的是 partition log。

問題二：我們走到這條 lane 的哪裡？

Kafka 在每個 partition 內為 record 指派 offset：

```
partition 0: ... 41, 42, 43
partition 1: ... 8, 9, 10
```

Offset 是 partition-local 的邏輯位置，不是全域時間，也不是整個 cluster 共用的流水號。它表示 record 在這條 log 裡的位置；它不表示業務操作何時完成。

Consumer 還有兩個容易混淆的進度：

- **consumer position**：這個 Consumer 下一次準備讀取或交付的位置；
- **committed offset**：Consumer group 保存的恢復 checkpoint，通常表示重新啟動時下一筆應從哪裡開始。

`poll` 取得資料後，consumer position 可以前進，但程式可能還沒完成資料庫寫入或 HTTP 呼叫。因此 position 不是完成證明。

Committed offset 也不是完成證明。它只代表應用程式告訴 Kafka：「未來恢復時，請從這裡開始。」Kafka 不會自行驗證先前的外部工作是否真的完成。

問題三：現在誰負責這條 lane？

多個 Consumer 使用相同的 `group.id` 時，形成普通的 **consumer group**。Kafka 會把 partitions 分配給 group members；在本文討論的普通 consumer group assignment 語意下，同一時間，一個 partition 會指派給該 group 中的一個 active Consumer。Kafka 的 Share Groups 是另一種 consumption model，不在本文主線。

因此：

- Consumer 少於 partitions：一個 Consumer 可能負責多條 lane。

- Consumer 多於 partitions：部分 Consumer 沒有 partition 可處理。
- 不同 consumer groups：可以各自完整讀取同一份 topic，並維持各自進度。

Kafka 協調的是 partition ownership 與 recovery position。它沒有因此取得外部資料庫、HTTP API 或背景執行緒的控制權。

See It in Failure | Ownership handoff 不等於舊工作停止

Consumer group 發生 ownership 轉移時，可以先用這條時間線理解：

```

Consumer A owns partition P
↓
A poll record, 開始外部工作
↓
A 停止 heartbeat / 失去 ownership
↓
Kafka 將 P 交給 Consumer B
↓
B 從 committed offset 恢復
↓
A 已啟動的舊外部工作仍可能完成

```

Kafka 可以讓舊 Consumer 失去對 partition 的 group ownership，也可以拒絕某些 stale group 操作；但它不能自動取消 A 已經送出的 HTTP request、SQL transaction 或背景執行緒。換句話說，**ownership transfer 不等於 old work disappeared**。這也是為什麼 recovery 不能只看「現在誰擁有 partition」，還要問上一個 owner 已經把哪些 effect 送出去了。

See It in Code | 先把進度邊界放回程式執行

下面不是完整 Java 範例，而是一段刻意簡化的 pseudocode：

```

records = consumer.poll(...)
for record in records:
    handle(record)          # external effect may already succeed

# —— failure window ——
# crash here: effect may exist, but committed offset has not advanced

consumer.commitSync()

```

真正要看的不是 API 語法，而是兩個時間點。`poll()` 之後，Consumer 的本地 position 可能已經前進；但在 `commitSync()` 完成之前，consumer group 保存的 committed offset 仍可能停在舊位置。

因此，如果 `handle(record)` 已經完成外部資料庫或 HTTP 操作，程式卻在 commit 前 crash，新的 Consumer 仍可能從舊 checkpoint 重新取得同一筆 record。這就是為什麼「讀過」、「處理過」與

「Kafka 已保存恢復進度」不能混成同一個狀態。

Checkpoint

在這裡，committed offset 表示 consumer group 恢復時「下一個要讀的位置」。某 Consumer 已取得 offsets 80–89，但只 committed offset 85，隨後 crash。接手的 Consumer 會從哪裡開始？哪些 records 可能再次出現？Kafka 能否知道其中哪些外部工作其實已經完成？

Deep Dive (選讀)

- Classic 與新版 consumer rebalance protocol
- Rebalance protocol internals、static membership
- High watermark、last stable offset

Chapter 4 — What Happens When Machines Fail?

Timeout 之後，我們到底知道什麼？

分散式系統最危險的句子之一是：

我收到錯誤，所以操作一定沒有成功。

考慮這條時間線：

```
Producer 發送 record
      ↓
Broker leader 寫入並完成必要複製
      ↓
Broker 回傳 acknowledgement
      ↓
回覆在網路中遺失
      ↓
Producer timeout
```

Producer 看到 timeout，但 record 可能已經存在，而且可能已對 Consumer 可見。

所以問題「寫入是否失敗？」的正確答案是：

單憑 timeout，我們不知道。

Producer 的 `acks` 設定決定成功回覆跨過哪個邊界：

- `acks=1`：leader 寫入自己的 log 後即可回覆。若它在複製前失效，已確認的 record 仍可能遺失。
- `acks=all`：leader 會等待目前 ISR 中的所有 replicas acknowledge；若目前 ISR 數量低於 `min.insync.replicas`，這筆 write 會失敗。這是 Kafka 可提供的最強 acknowledgement 設定，但仍不代表所有 assigned replicas、disk fsync 或下游業務處理都已完成。

當結果不確定時，Producer 可能 retry。問題是：第一次其實成功了嗎？如果成功，再送一次會不會形成 duplicate？

Kafka 的 **idempotent producer** 使用 producer identity 與 partition sequence state，辨認同一批 protocol retry，避免它被重複 append。這是一個重要但有限的保護：

- 它保護符合條件的 Kafka producer retry。
- 它不會辨認應用程式稍後自行建立的另一筆 `send` 是否代表同一個商業操作。
- 它更不會替外部 API 或資料庫去重。

Consumer 端也有相同類型的不確定性：

```
Consumer 讀取 record
  ↓
外部 effect 成功
  ↓
Consumer crash
  ↓
offset 尚未 commit
  ↓
另一個 Consumer 重新讀到 record
```

Kafka 知道 committed offset 尚未前進，所以會重新交付 record；它不知道先前的郵件、扣款或資料庫更新是否已完成。反過來，若先 commit offset、再執行外部 effect，兩者之間 crash，則可能永遠跳過那次 effect。

失敗不是單一布林值。必須分別問：哪個 actor 做到了哪一步？哪些狀態已保存？誰看得見？誰只看見 timeout？還有哪些結果無法確定？

See It in Config | 先認得三個最常碰到的名字

這一章不需要先學 tuning，但之後看到 Producer 設定時，至少要能把名稱和問題對回來：

- `acks`：成功回覆需要跨過哪個 broker-side acknowledgement / replication 邊界。
- `enable.idempotence`：讓 Producer 使用 identity 與 sequence state 抑制符合條件的 protocol retry 重複 append；它不是 business-level idempotency。
- `retries`：Client 在可重試錯誤下是否可能重送請求。它只回答「會不會再嘗試」，不回答「這次 retry 在商業語意上是否安全」。

這三個設定不是三個獨立魔法開關。理解它們之前，先回到 failure timeline：前一次 attempt 到底可能做到哪裡？目前留下了什麼 evidence？

Checkpoint

Producer timeout 時，請分別指出：它已知什麼、不知道什麼，以及「直接再建立一次新的 send」為什麼不一定等同於 Kafka 能安全去重的 protocol retry？

Deep Dive (選讀)

- ISR、Eligible Leader Replicas
- Producer sequence、epoch 與 fencing
- `min.insync.replicas`

Chapter 5 — Three Dangerous Kafka Misunderstandings

誤解一：Partition order 就是 global order

Kafka 的順序存在於 partition 內，不存在於整個 topic 的所有 partitions 之間。

如果訂單 A 在 partition 0、訂單 B 在 partition 1，offset 無法跨 partition 比較。即使兩筆 record 都帶 timestamp，timestamp 也可能受到時鐘、網路延遲與 Producer 行為影響，不能自動變成 Kafka 的全域順序保證。

安全的說法是：

Kafka 能為被放進同一 partition 的 records 建立 log order。

不能把這句話擴張成：

Kafka 知道整個系統所有事件真正發生的先後。

誤解二：Kafka success 就是 business completion

Kafka 回傳的每一種成功，都只證明一件有邊界的事。

技術證據	它能證明什麼	它不能證明什麼
Producer acknowledgement	Record 已跨過設定要求的 broker 寫入 / 複製條件	Consumer 已讀取、訂單已完成
Committed offset	Consumer group 保存了新的恢復位置	HTTP、資料庫或其他 effect 已完成
Consumer 收到 record	該 record 已被交付給這次處理	處理一定成功，或未來不會重送

看到技術成功時，應先問：「這是誰對哪一段狀態負責的證據？」而不是直接把它翻譯成商業成功。

誤解三：Idempotence、transactions、replay 等於 exactly-once-everything

這三個機制都很有價值，但邊界不同：

- Producer idempotence 防止特定 Kafka retry 形成重複 append。
- Kafka transactions 可以共同協調 Kafka records 與 Kafka consumer offsets。

- `replay` 允許 Consumer 再讀取仍被保留的資料。

它們都不等於「任意商業行為只發生一次」。

Kafka transaction 不會自動把一般 HTTP API 或外部資料庫納入同一個 atomic boundary。即使 Kafka 內部的 records 與 offsets 一起 commit，外部 effect 仍可能需要另一套設計。

Replay 也不是時光機。Retention 可以刪除舊資料；compaction 可以移除同一 key 的舊值；tombstone 與 schema 等必要資訊也可能不再存在。能否重建狀態，取決於需要的 evidence 是否仍被保留並且可以解讀。

Checkpoint

有人說：「我們已使用 Kafka transaction，所以資料庫扣款、Kafka output record 與 consumer offset 一定會 exactly once。」這句話隱藏了哪個尚未成立的 atomicity 假設？

Deep Dive (選讀)

- Kafka transaction coordinator、transaction marker、LSO
- Transactional consume-transform-produce
- Outbox / inbox patterns

Chapter 6 — From Kafka User to Kafka Contributor

從一個 **guarantee** 走到 **source**、**test**、**issue**

Contribution Ready 不等於先讀完整個 Kafka repository。

Kafka 橫跨 client、broker、storage、replication、group coordination、transactions 與 KRaft controller。如果一開始就從整條 broker request path 往下讀，很容易看到大量類別，卻不知道哪段行為才是自己要驗證的 contract。

更有效的入口是一項有邊界的 guarantee：

```
公開 contract
  ↓
設定與限制
  ↓
focused test
  ↓
internal state machine
  ↓
issue 或最小修正
```

第一次進 source，不需要直接挑最複雜的狀態機。可以先走一條比較輕的路徑：

入門路線 | 先看一個容易對回公開行為的 client internal

```
KafkaProducer Javadoc
  ↓
ProducerConfig
  ↓
KafkaProducerTest / focused producer test
  ↓
RecordAccumulator
```

這條路徑可以先觀察 Producer 如何把公開設定與 record batching 的內部行為接起來。重點不是一次讀懂 `RecordAccumulator` 全部細節，而是練習把一項公開 contract 對到一個 focused test，再找到負責那段狀態的 implementation。

進階路線 | 當你真的研究 retry、idempotence 或 transaction

```
KafkaProducer Javadoc
    ↓
ProducerConfig
    ↓
TransactionManagerTest
    ↓
TransactionManager
```

`TransactionManager` 涉及 producer identity、sequence、epoch、fencing 與 transaction state；它很有價值，但不需要成為第一次讀 Kafka source 的唯一入口。

Javadoc 告訴你公開承諾；configuration 說明哪些行為依賴設定；focused test 把承諾轉成可執行的案例；internal state machine 才是最後要讀的實作。

面對一個疑似 bug，可以依序問：

1. 公開文件實際承諾了什麼？
2. 這次行為涉及哪些實體 actor？
3. 預期發生哪個 state transition？
4. Failure 可以在哪一步中斷？
5. 中斷後留下哪些 persisted evidence？
6. 哪個 focused test 最接近這項 contract？
7. 只有到這裡，才進入 implementation 尋找原因或修改點。

這個流程能避免兩種常見錯誤：看到 exception 就直接猜 root cause；看到可疑程式碼就修改，卻沒有先確認它是否違反公開保證。

Kafka 目前以 **JIRA** 追蹤 issue，以 **GitHub** 進程式碼與 Pull Request review。影響公開 API、protocol、configuration 或重大行為的變更，通常需要走 **KIP** 討論與投票；小型 bug fix 並不會因為修改 Kafka 就自動需要 KIP。

第一次貢獻可以只是：

- 一個可重現的 failure timeline；
- 一個能暴露 contract 差異的 focused test；
- 一份把「觀察」與「推測原因」清楚分開的調查；
- 一個範圍很小、證據充分的修正。

不要只因 JIRA 帶有 `newbie` label 就假設它仍然適合。還要確認 issue 是否仍有效、是否已有 PR、是否能在目前版本重現，以及修改範圍是否真的有限。

Contribution Ready does not mean knowing all of Kafka. It means being able to trace one bounded behavior from guarantee to evidence to implementation.

Checkpoint

如果 issue 聲稱「Producer timeout 造成 duplicate」，在打開 implementation 前，你會先建立哪些 evidence？至少應確認哪些公開承諾、相關設定、failure timeline 與 focused test？

Deep Dive (選讀)

- Kafka protocol message schemas
- Broker、storage 與 KRaft controller 的 source topology
- Integration tests 與 system tests

技術依據與延伸閱讀

Chapter 1–2 : 存在理由、partition 與 ordering

- [Apache Kafka 4.3 Introduction](#)
- [Apache Kafka 4.3 Design](#)
- [Producer Configuration](#)
- [Basic Kafka Operations](#)

Chapter 3 : offset、consumer progress 與 group ownership

- [KafkaConsumer Javadoc](#)
- [Consumer Configuration](#)
- [Consumer Rebalance Protocol](#)
- [KIP–848: The Next Generation of the Consumer Rebalance Protocol](#)

Chapter 4–5 : acknowledgement、failure、transactions 與 replay

- [KafkaProducer Javadoc](#)
- [Apache Kafka Design : Replication and Transactions](#)
- [KIP–98: Exactly Once Delivery and Transactional Messaging](#)
- [Topic Retention and Compaction Configuration](#)

Chapter 6 : source 與 contribution workflow

- [Apache Kafka Developer Guide](#)
- [Contributing Code Changes](#)
- [Kafka Improvement Proposals](#)
- [KafkaProducer source area — Kafka 4.3.1](#)
- [KafkaProducerTest — Kafka 4.3.1](#)
- [RecordAccumulator — Kafka 4.3.1](#)
- [TransactionManagerTest — Kafka 4.3.1](#)
- [TransactionManager — Kafka 4.3.1](#)

方法論來源

本教材的 first-principles 教學方法，源自 **Yen-Hua Chen** 在 **Streaming System + Compass** 中長期記錄的工程推理實踐，包括 ADR、postmortem、reasoning notes 與系統設計分析。

Kafka track 並不是將 Compass 的架構直接套用到 Kafka。教材中的推理模式，都再以 Apache Kafka 官方文件、Javadoc、KIP、source code 與 tests 獨立驗證；若兩者的物理機制不相符，相關類比即不採用。

這段 provenance 描述的是本教材 teaching methodology 與 framing 的來源，並不表示 Kafka 的架構或設計源自 Compass。

Streaming System + Compass — Yen-Hua Chen

Source: <https://github.com/hikaru-212/streaming-system-compass>

Documentation license: CC BY 4.0

作者短註

- 主線刻意省略：KRaft internals、ISR / ELR 細節、完整 transaction protocol、LSO 演算法、Share Consumer、Kafka Streams / Connect、security、schema evolution 與管理操作。
- 本版新增少量 executable bridge：以 pseudocode、關鍵 config 名稱與兩條 source-reading 路徑，把抽象概念對回實際工程表面，但刻意不擴張成 API 教學。
- v1.1 進一步把兩個 failure boundary 視覺化：`handle → commit` 的 crash window，以及 consumer ownership 轉移後舊外部工作仍可能完成的 handoff window。
- 擴充版適合加入：可操作的 failure labs、rebalance 深入分析、external database 的 outbox / inbox patterns、compaction rebuild 條件、更多 source/test 閱讀路線，以及經實際驗證的 newcomer issue。